

Phase 1: Project Planning

Step 1: Create the project

Create a GitHub repository.

```
car-dealership-inventory
```

Project structure:

```
car-dealership-inventory/  
|  
├─ backend/  
├─ frontend/  
├─ README.md  
├─ PROMPTS.md  
└─ .gitignore
```

Commit:

```
git commit -m "chore: initialize project structure"
```

Phase 2: Backend

Step 2: Ask Antigravity to generate FastAPI boilerplate

Prompt:

Create a FastAPI backend project for a Car Dealership Inventory System. Use Python 3.12, FastAPI, SQLAlchemy ORM, Alembic migrations, PostgreSQL, Pydantic v2, JWT authentication, Passlib for password hashing, and Pytest for testing. Organize the project into routers, models, schemas, services, database, authentication, and tests folders following clean architecture and SOLID principles.

Commit

```
chore: setup FastAPI project
```

Step 3: Database Connection

Prompt

Configure PostgreSQL connection using SQLAlchemy and environment variables. Add Alembic migration support. Create Base model and database session dependency.

Commit

```
feat: configure PostgreSQL database
```

Step 4: User Model

Prompt

Create SQLAlchemy User model with id, username, email, hashed_password, role (user/admin), created_at. Create matching Pydantic schemas.

Write tests first.

Commit

```
test: add user model tests
```

Then

```
feat: implement user model
```

Step 5: Authentication

Prompt

Implement JWT authentication including Register and Login APIs. Hash passwords using Passlib bcrypt. Return JWT access token after successful login. Protect routes using OAuth2PasswordBearer.

Endpoints

```
POST /api/auth/register
```

```
POST /api/auth/login
```

Commit

```
test: authentication endpoints
```

```
feat: JWT authentication
```

Step 6: Vehicle Model

Prompt

Create Vehicle model with id, make, model, category, price, quantity, created_at and updated_at. Generate SQLAlchemy model, schemas and migration.

Commit

```
feat: vehicle model
```

Step 7: CRUD APIs

Prompt

Create REST APIs for vehicles:

Add Vehicle

List Vehicles

Update Vehicle

Delete Vehicle (Admin only)

Commit

```
feat: vehicle CRUD
```

Step 8: Search API

Prompt

Implement GET /api/vehicles/search supporting make, model, category, minimum price and maximum price filters. Support multiple filters simultaneously.

Commit

feat: vehicle search

Step 9: Purchase API

Prompt

Implement Purchase API that decreases vehicle quantity by one. Return validation error if quantity becomes negative.

Commit

feat: purchase vehicle

Step 10: Restock API

Prompt

Implement Restock API. Only Admin users can increase stock quantity.

Commit

feat: restock inventory

Step 11: Testing

Prompt

Write comprehensive Pytest test cases for authentication, CRUD, search, purchase and restock APIs. Cover successful and failure scenarios.

Commit

test: backend API coverage

Phase 3: Frontend

Step 12: React Project

Prompt

Create a React application using Vite. Install Tailwind CSS, React Router and Axios. Organize project into components, pages, services, hooks and context folders.

Commit

chore: setup React frontend

Step 13: Login & Register

Prompt

Create responsive Login and Register pages using Tailwind CSS. Connect to FastAPI authentication APIs using Axios. Store JWT token securely and redirect after login.

Commit

feat: authentication UI

Step 14: Dashboard

Prompt

Create dashboard showing all vehicles in responsive cards. Display make, model, category, price and quantity.

Commit

feat: dashboard

Step 15: Search

Prompt

Add search filters for make, model, category and price range. Call backend search API

while typing.

Commit

```
feat: vehicle filtering
```

Step 16: Purchase Button

Prompt

Add Purchase button for every vehicle. Disable button when quantity equals zero.
Refresh inventory after purchase.

Commit

```
feat: purchase UI
```

Step 17: Admin Panel

Prompt

Create Admin Dashboard with forms to Add, Update and Delete vehicles. Protect page so only admin users can access it.

Commit

```
feat: admin dashboard
```

Phase 4: Deployment

Backend

Prompt

Prepare FastAPI backend for deployment on Render. Configure environment variables, CORS, production settings and PostgreSQL.

Frontend

Prompt

Prepare React frontend for deployment on Vercel. Configure environment variables and production API URL.

Commit

chore: production deployment

Phase 5: Documentation

README.md

Prompt

Write a professional README including project overview, technologies, installation, API endpoints, screenshots, deployment instructions, testing guide and project architecture.

PROMPTS.md

Since the project requires AI transparency, keep a running log of every prompt you give Antigravity (or ChatGPT) and paste them into `PROMPTS.md`. Also include a section in `README.md` describing which AI tools you used and how they helped.

Recommended Development Order

1. ☒ FastAPI setup
2. ☒ PostgreSQL
3. ☒ User authentication
4. ☒ JWT security
5. ☒ Vehicle model
6. ☒ CRUD APIs
7. ☒ Search
8. ☒ Purchase
9. ☒ Restock
10. ☒ Backend tests

11. ☒ React setup
12. ☒ Login/Register
13. ☒ Dashboard
14. ☒ Search & filters
15. ☒ Purchase flow
16. ☒ Admin panel
17. ☒ Responsive UI
18. ☒ Deploy backend
19. ☒ Deploy frontend
20. ☒ Documentation

This incremental workflow will produce a clean commit history, make it easier to follow TDD, and keep the project manageable while satisfying the assignment requirements.